CS 331 – Software Engineering Lab

Assignment 8

Smart Ride Demand Anticipation & Pricing Optimizer (SRDAPO)

Field	Details
Subject	CS 331 – Software Engineering Lab
Assignment	Assignment 8
Topic	Data Access Layer + White Box & Black Box Testing
Group Members	Pranita Mahajan (2301151)
	Pulkit (2301165)
	Shivang Sharma (2301203)

Project Overview

SRDAPO is an end-to-end **intelligent ride demand forecasting and pricing optimisation system** inspired by Ola/Uber platforms. It predicts ride demand at a zone level using SARIMA, XGBoost, and LSTM models, recommends dynamic surge pricing, and exposes secure JWT-authenticated REST APIs with MongoDB as the persistence layer.

Architecture Overview

Layer	Component	Technology
Machine Learning	Demand forecasting (SARIMA / XGBoost / LSTM)	Python, Statsmodels, XGBoost, TensorFlow/Keras
Data Access Layer	MongoDB store + repository + cache fallback	PyMongo, Python
Backend / API	REST endpoints with JWT authentication	FastAPI, python-jose
Scheduler	Automated hourly predictions	APScheduler
Frontend	Heatmaps, charts, zone visualisation	React.js, Tailwind CSS, Leaflet

PART A – DATA ACCESS LAYER

A.1 What is a Data Access Layer?

The **Data Access Layer (DAL)** is an architectural component that abstracts and encapsulates all database operations from the rest of the application. It acts as a bridge between the business/application logic and the underlying database, exposing clean methods (insert, query, update, delete) while hiding the complexity of query languages, connection management, and schema details.

Concern	Without DAL	With DAL
DB coupling	Business logic contains raw queries	Business logic calls repository methods
Testability	Hard – requires live DB	Easy – mock the store interface
Portability	Switching DB rewrites the whole app	Only DAL code changes
Consistency	Query patterns duplicated everywhere	Centralised, reusable queries

A.2 Database Design (MongoDB)

MongoDB was chosen for its flexible document model, which suits the semi-structured nature of ride predictions (different ML models produce different feature sets). Three collections are created:

Collection 1 – predictions

Field	Type	Description
_id	ObjectId	Auto-generated primary key
model_name	String	Name of ML model (SARIMA / XGBoost / LSTM / RandomForest)
zone_id	Integer	Numeric identifier of the city zone
predicted_demand	Float	Forecasted number of ride requests
surge_factor	Float	Recommended surge multiplier for pricing
created_at	DateTime	UTC timestamp of prediction generation

Index: `idx_model_zone` – Compound index on (model_name, zone_id) for fast filtered queries.

Collection 2 – users

Field	Type	Description
_id	ObjectId	Auto-generated primary key
email	String (unique)	User login identifier
hashed_password	String	Bcrypt-hashed password
role	String	admin | driver | analyst
created_at	DateTime	UTC account creation timestamp

Index: `uniq_email` – Unique ascending index on email to enforce uniqueness and fast lookups.

Collection 3 – test_cases

Field	Type	Description
_id	ObjectId	Auto-generated primary key
suite	String	Test suite name (WhiteBox / BlackBox)
type	String	WB or BB
test_id	String	E.g. WB-1, BB-2
result	String	PASS / FAIL
run_at	DateTime	Test execution timestamp

Index: `idx_suite_type` – Compound index on (suite, type) for filtered test-result queries.

A.3 DAL Component Implementation

The DAL is implemented in `backend/api/dal.py` using two classes:

MongoPredictionStore (Low-level MongoDB wrapper)

Handles raw MongoDB operations: connecting, collection creation, index creation, and CRUD. Key methods:

- connect() / disconnect() – manages MongoClient lifecycle
- ensure_database_objects() – creates missing collections and indexes (idempotent)
- insert_prediction() / find_predictions() – predictions CRUD
- insert_user() / find_user_by_email() – users CRUD
- insert_test_result() – stores test execution results

```
class MongoPredictionStore:
    REQUIRED_COLLECTIONS = ['predictions', 'users', 'test_cases']

    def connect(self) -> bool:
        self.client = MongoClient(self.uri, serverSelectionTimeoutMS=3000)
        self.client.admin.command('ping')
        self.db = self.client[self.db_name]
        return True

    def ensure_database_objects(self):
        existing = self.db.list_collection_names()
        for coll in self.REQUIRED_COLLECTIONS:
            if coll not in existing:
                self.db.create_collection(coll)
```

PredictionRepository (High-level Repository with cache fallback)

Implements the **Repository Pattern**: exposes clean `list_predictions()` and `save_prediction()` methods. When MongoDB is unavailable or raises an error, it transparently falls back to an in-process list cache so the API continues working.

```
def list_predictions(self, model_name=None, zone_id=None):
    try:
        if self._connected and self.store.is_connected():
            docs = self.store.find_predictions(model_name, zone_id)
            for d in docs: d['source'] = 'mongodb'
            return docs
        else:
            return self._from_cache(model_name)
    except PyMongoError as exc:
        logger.error('Falling back to cache: %s', exc)
        return self._from_cache(model_name)
```

A.4 Database Initialisation Script

The script `backend/api/init_db.py` creates all collections and indexes, and seeds sample prediction and user documents.

Sample seed data inserted:

model_name	zone_id	predicted_demand	surge_factor
SARIMA	1	142	1.2
XGBoost	2	87	1.0
LSTM	3	210	1.8
RandomForest	1	130	1.1
RandomForest	4	65	0.9

PART B – WHITE BOX & BLACK BOX TESTING [20 Marks]

B.1 Testing Theory

Aspect	White Box Testing	Black Box Testing
Definition	Tester has full knowledge of internal code/logic	Tester treats the system as a black box; only I/O matters
Also Known As	Glass Box / Structural Testing	Functional / Behavioural Testing
Focus	Internal code paths, branches, conditions	External behaviour, API contracts, user stories
Knowledge Required	Source code access required	Specification / API docs only
Who Performs	Developers	QA Engineers / End-users
Goal	Ensure every code path is exercised	Ensure correct output for given input
Techniques	Statement, branch, path coverage	Equivalence partitioning, boundary analysis
Applied To (SRDAPO)	DAL internals (collection creation, index logic, branching)	FastAPI endpoint behaviour (auth, response shape)

B.2 White Box Test Cases

Five white box tests exercise **internal DAL code paths** in `MongoPredictionStore` and `PredictionRepository`. Each test uses **unittest.mock** to simulate MongoDB state without requiring a live database.

ID	Component Under Test	Test Description	Branch / Path Exercised	Expected Result	Status
WB-1	MongoPredictionStore .ensure_database_objects()	DB has no collections → all three must be created	create_collection() branch (collection not in existing)	created_collections contains 'predictions', 'users', 'test_cases'	PASS
WB-2	MongoPredictionStore .ensure_database_objects()	Verify index creation fires for all collections	Index creation calls for idx_model_zone, uniq_email, idx_suite_type	created_indexes list contains all three index names	PASS
WB-3	PredictionRepository .list_predictions()	MongoDB connected, documents available	if self._connected and self.store.is_connected(): → True branch	Returns docs with source='mongodb'	PASS
WB-4	PredictionRepository .list_predictions()	MongoDB NOT connected (_connected=False)	else: branch → _from_cache()	Returns cache docs with source='cache'	PASS
WB-5	PredictionRepository .list_predictions()	find_predictions() raises PyMongoError	except PyMongoError: fallback branch	Exception caught; returns cache docs with source='cache'	PASS

B.3 Black Box Test Cases

Two black box tests use the **FastAPI TestClient** to call the `GET /predictions` endpoint as an external consumer would, with no knowledge of internal DAL implementation.

ID	Endpoint	Pre-condition	Test Input	Expected Output	Status
BB-1	GET /predictions	No authentication token	HTTP GET /predictions (no Authorization header)	HTTP 401 Unauthorized (no valid token supplied)	PASS
BB-2	GET /predictions ?model_name= RandomForest	Repository has one matching document; valid JWT obtained	HTTP GET with Bearer token and model_name filter	HTTP 200 OK; JSON list with model_name, zone_id, predicted_demand, surge_factor fields	PASS

B.4 Test Execution Results

All 7 tests were executed using pytest. Output:

```
$ python -m pytest backend/tests/test_dal.py -v

collected 7 items

test_dal.py::TestWhiteBox_EnsureDatabaseObjects::test_WB1 ... PASSED  [14%]
test_dal.py::TestWhiteBox_EnsureDatabaseObjects::test_WB2 ... PASSED  [28%]
test_dal.py::TestWhiteBox_ListPredictions::test_WB3       ... PASSED  [42%]
test_dal.py::TestWhiteBox_ListPredictions::test_WB4       ... PASSED  [57%]
test_dal.py::TestWhiteBox_ListPredictions::test_WB5       ... PASSED  [71%]
test_dal.py::TestBlackBox_PredictionsEndpoint::test_BB1   ... PASSED  [85%]
test_dal.py::TestBlackBox_PredictionsEndpoint::test_BB2   ... PASSED  [100%]

======================== 7 passed in 1.40s ========================
```

Test ID	Type	Component / Endpoint	Result
WB-1	White Box	MongoPredictionStore.ensure_database_objects() – collection creation	PASS
WB-2	White Box	MongoPredictionStore.ensure_database_objects() – index creation	PASS
WB-3	White Box	PredictionRepository.list_predictions() – MongoDB branch	PASS
WB-4	White Box	PredictionRepository.list_predictions() – cache branch	PASS
WB-5	White Box	PredictionRepository.list_predictions() – PyMongoError branch	PASS
BB-1	Black Box	GET /predictions – no authentication token	PASS
BB-2	Black Box	GET /predictions – authenticated + filter	PASS